

12 강

머신러닝

커널 기법 2

통계·데이터과학과 박준희



✦ 학습목차

- 1 Soft-Margin SVM
- 2 Soft-Margin SVM 실습
- 3 서포트 벡터 회귀 (SVR)
- 4 서포트 벡터 회귀 실습



1

Soft-Margin SVM



1

Soft-Margin SVM

● Hard-Margin SVM의 한계

- 데이터가 linearly separable해야함
- 현실 데이터 : 노이즈나 클래스 간 중첩 존재
- 슬랙 변수 : 일부 샘플이 마진을 침범하거나 오분류 허용

1

Soft-Margin SVM

● 슬랙 변수($\xi_n, n = 1, \dots, N$)

- 각 데이터 샘플 x_n 에 대응하는 ξ_n ($n = 1, \dots, N$)은 다음을 정량화하는 변수이다:
 - 마진을 얼마나 침범하는지
 - 혹은 오분류가 어느 정도로 발생했는지
- $\xi_n = 0$ 이면 해당 샘플은 마진을 전혀 침범하지 않고 (Hard-Margin과 동일),
- $\xi_n > 0$ 이면 샘플이 마진 안쪽에 있거나, 심지어 다른 클래스 쪽으로 넘어간 상태(오분류)를 의미한다.

1

Soft-Margin SVM

● Hard-Margin SVM 과의 비교

- **Hard-Margin:** 모든 샘플이 초평면 바깥쪽(자신의 클래스 쪽)에 있으며, 침범하지 않는다 ($\xi_n = 0$).
- **Soft-Margin:** 일부 샘플이 어쩔 수 없이 마진을 침범($\xi_n > 0$), 혹은 오분류($\xi_n > 1$)되더라도, 전체 모델의 일반화 성능과 마진 확보의 균형을 잡는다.

1

Soft-Margin SVM

● Soft-Margin SVM(C-SVM)의 장점

- **현실성**: 노이즈나 중첩이 있는 데이터에 대해 실제로 사용 가능
- **오분류 허용**: 모델이 약간의 오분류를 허용함으로써 오버피팅을 피하고, 복잡한 데이터에도 잘 대응
- **Hard-Margin 확장**: $C \rightarrow \infty$ 로 가면 Hard-Margin과 유사해짐(이론적으로).

1 Soft-Margin SVM

● 프라이멀 문제

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n, \\ \text{subject to} \quad & t_n (\mathbf{w}^T \phi(x_n) + b) \geq 1 - \xi_n, \\ & \xi_n \geq 0 \quad (\text{for all } n = 1, \dots, N) \end{aligned}$$

- $\mathbf{w}, b$: 결정 초평면의 마진을 정의하는 파라미터
- $\xi = [\xi_1, \xi_2, \dots, \xi_N]^T$: 각 샘플이 분류 초평면의 마진을 침범하는 정도 (오분류까지 포함)
- $C > 0$: **trade-off constant**로서 마진 크기와 오분류(침범) 비용 간의 트레이드오프를 사용자가 직접 조절

1 Soft-Margin SVM

● 라그랑지안

프라이멀 문제의 부등식 제약조건들을 반영하여 다음과 같이 **제약조건 ≤ 0** 형태 및, **라그랑주 승수 ≥ 0** 형태인 라그랑지안을 정의한다.

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) \\ = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n + \sum_{n=1}^N \alpha_n \left[1 - \xi_n - t_n (\mathbf{w}^T \phi(x_n) + b) \right] - \sum_{n=1}^N \mu_n \xi_n \end{aligned}$$

1 Soft-Margin SVM

● 라그랑지안

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{n=1}^N \alpha_n t_n \phi(x_n),$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{n=1}^N \alpha_n t_n = 0,$$

$$\frac{\partial \mathcal{L}}{\partial \xi_n} = 0 \implies \alpha_n = C - \mu_n \in [0, C], \quad n = 1, \dots, N.$$

1 Soft-Margin SVM

● 듀얼 문제

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m \phi(x_n)^T \phi(x_m), \\ \text{subject to} \quad & \sum_{n=1}^N \alpha_n t_n = 0, \quad \alpha_n \in [0, C] \quad (n = 1, \dots, N). \end{aligned}$$

1 Soft-Margin SVM

● KKT 조건

1. Stationarity 조건

$$\mathbf{w} = \sum_{n=1}^N \alpha_n t_n \phi(x_n), \quad \sum_{n=1}^N \alpha_n t_n = 0, \quad \alpha_n \in [0, C], \quad n = 1, \dots, N.$$

2. Primal feasibility (프라이멀 타당성)

$$1 - \xi_n - t_n (\mathbf{w}^T \phi(x_n) + b) \leq 0, \quad \xi_n \geq 0, \quad n = 1, \dots, N.$$

1 Soft-Margin SVM

● KKT 조건

3. Dual feasibility (듀얼 타당성)

$$\alpha_n \geq 0, \quad \mu_n \geq 0, \quad n = 1, \dots, N.$$

4. Complementary slackness (상호 보완성)

$$\alpha_n [1 - \xi_n - t_n (\mathbf{w}^T \phi(x_n) + b)] = 0, \quad \mu_n \xi_n = 0, \quad n = 1, \dots, N.$$

2

Soft-Margin SVM

실습



2 Soft-Margin SVM 실습

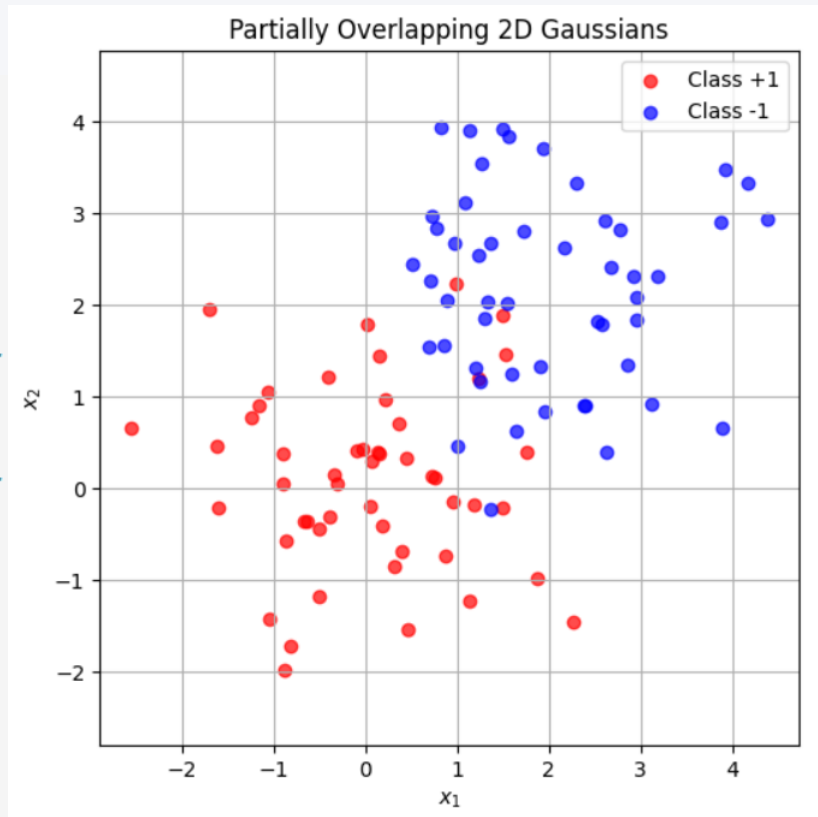
● 겹치는 이변량 정규분포 데이터

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# -----
# 1) 예시 데이터: 겹치는 두 이변량 정규분포 샘플
# -----

np.random.seed(0)

mean_plus = [0, 0]
mean_minus = [2, 2]
cov = [[1, 0],
        [0, 1]]
```



2 Soft-Margin SVM 실습

● RBF 커널 및 커널 행렬

```
def rbf_kernel(x1, x2, gamma):  
    return np.exp(-gamma * np.sum((x1 - x2)**2))  
  
def compute_kernel_matrix(X, gamma):  
    N = len(X)  
    K = np.zeros((N, N))  
    for i in range(N):  
        for j in range(N):  
            K[i, j] = rbf_kernel(X[i], X[j], gamma)  
    return K
```


2 Soft-Margin SVM 실습

● 결정 함수

```
def decision_function(alpha, b, X_train, t_train, gamma, X_test):  
    """  
     $f(x) = \sum_n \alpha_n * t_n * K(x_n, x) + b$   
    """  
    out = []  
    for x in X_test:  
        val = b  
        for i in range(len(X_train)):  
            if alpha[i] > 1e-8:  
                val += alpha[i] * t_train[i] * rbf_kernel(X_train[i], x, gamma)  
        out.append(val)  
    return np.array(out)
```

2 Soft-Margin SVM 실습

● 예측함수, 결정 경계 시각화 함수

```
def predict(alpha, b, X_train, t_train, gamma, X_test):
    vals = decision_function(alpha, b, X_train, t_train, gamma, X_test)
    return np.sign(vals)

def plot_decision_boundary(alpha, b, X, t, gamma, title="RBF SVM"):
    x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
    y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )
    grid_points = np.c_[xx.ravel(), yy.ravel()]
    Z = predict(alpha, b, X, t, gamma, grid_points)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.bwr)
```

출처:파이썬 프로그램 실습내용 중

2 Soft-Margin SVM 실습

● 듀얼 목적 함수

```
def dual_objective_soft(alpha, t, K):  
    """  
    Dual objective:  
        maximize  $W(\alpha) = \sum(\alpha_n) - 1/2 \sum_{n,m} \alpha_n \alpha_m t_n t_m K(n,m)$   
    We'll minimize the negative of  $W(\alpha)$ .  
    """  
    term1 = 0.5 * np.sum((alpha * t)[: , None] * (alpha * t)[None, :] * K)  
    term2 = np.sum(alpha)  
    return term1 - term2
```

2 Soft-Margin SVM 실습

- RBF SVM 함수: $0 \leq \alpha_n \leq C$ 의 제약조건이 추가

```
def fit_svm_rbf_soft(X, t, gamma, C):
    """
    Solve the Soft-Margin SVM dual problem:
        0 <= alpha_n <= C
        sum_n alpha_n t_n = 0
    """
    N = len(X)
    K = compute_kernel_matrix(X, gamma=gamma)

    alpha0 = np.zeros(N)

    def eq_constraint(alpha):
        return np.dot(alpha, t)

    constraints = [{'type': 'eq', 'fun': eq_constraint}]
    bounds = [(0, C)] * N # 소프트 마진: alpha_n <= C
```

출처:파이썬 프로그램 실습내용 중

2 Soft-Margin SVM 실습

- RBF SVM 함수: $0 \leq \alpha_n \leq C$ 의 제약조건이 추가

```
res = minimize(  
    fun=dual_objective_soft,  
    x0=alpha0,  
    args=(t, K),  
    bounds=bounds,  
    constraints=constraints,  
    options={'maxiter': 1000, 'disp': True}  
)  
alpha_opt = res.x
```

2 Soft-Margin SVM 실습

● b 계산하기 : 서포트 벡터들을 이용하여 평균

```
# b 계산 (KKT)
sv_indices = np.where((alpha_opt > 1e-8) & (alpha_opt < C - 1e-8))[0]
if len(sv_indices) == 0:
    b_opt = 0.0
else:
    b_list = []
    for i in sv_indices:
        sum_j = np.sum(alpha_opt * t * K[i, :])
        b_i = t[i] - sum_j
        b_list.append(b_i)
    b_opt = np.mean(b_list)

return alpha_opt, b_opt
```

출처:파이썬 프로그램 실습내용 중

2 Soft-Margin SVM 실습

● 예시 실행

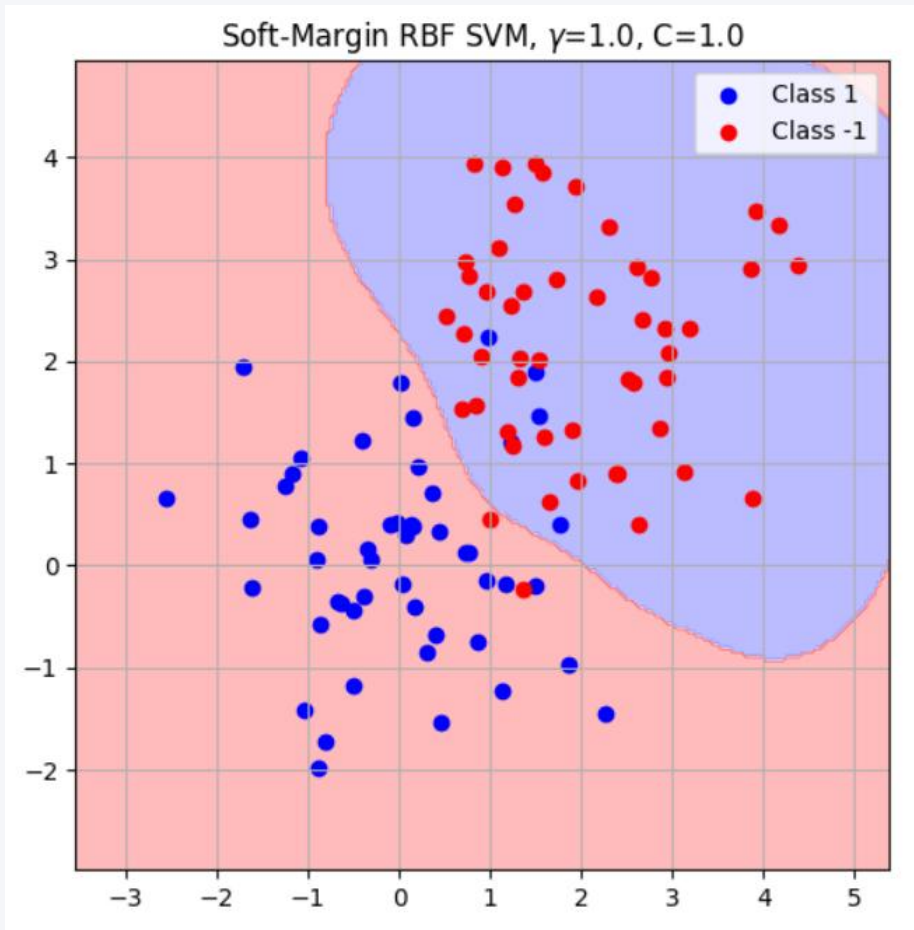
```
gamma = 1.0
C = 1.0

alpha, b = fit_svm_rbf_soft(X, t, gamma=gamma, C=C)
print("\n alpha:", alpha)
print("b:", b)

plt.figure(figsize=(6,6))
plot_decision_boundary(alpha, b, X, t, gamma,
                        title=rf"Soft-Margin RBF SVM, $\gamma$={gamma}, C={C}")
plt.show()
```

2 Soft-Margin SVM 실습

● 결과



출처:파이썬 프로그램 실습내용 중

3

서포트 벡터 회귀



● Support Vector Regression (SVR)

- **SVM for Regression:** 기존 SVM이 분류 문제에 특화된 반면, **SVR**은 연속 값 예측(회귀) 문제로 확장한 모델이다.
- **핵심 아이디어:** 회귀 예측 선(또는 초평면) 주변에 ϵ -**tube**를 설정하고, 그 내부의 오차는 무시(ϵ -insensitive)하며, 튜브 바깥으로 벗어난 샘플에 대해서만 비용($\xi, \hat{\xi}$)을 부여한다.

3 서포트 벡터 회귀

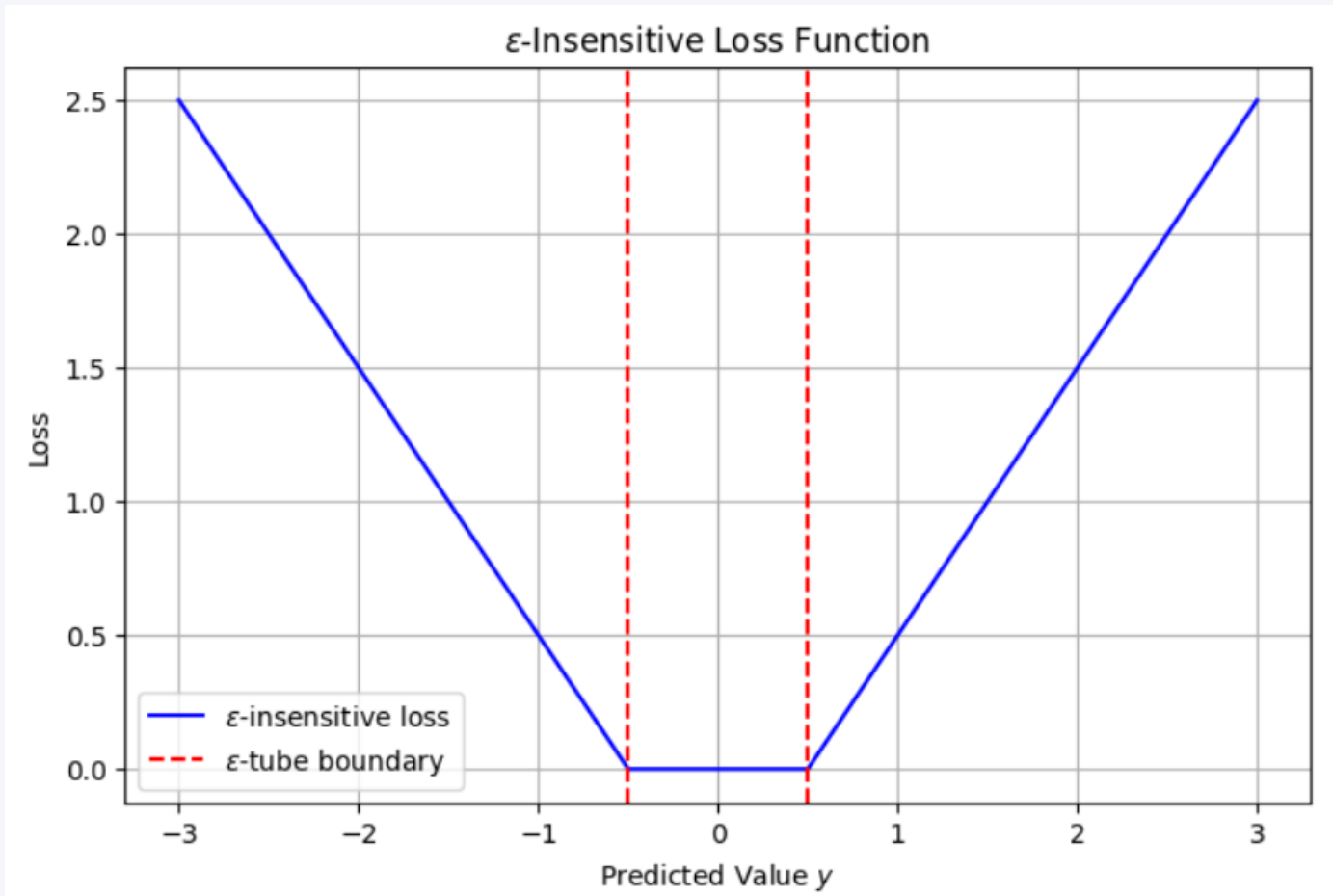
● ε -insensitive 손실 함수

$$|t_n - y(x_n)|_\varepsilon = \max(|t_n - y(x_n)| - \varepsilon, 0),$$

- 예측값 $y(x_n)$ 이 ε -**tube** 내부에 있으면 손실이 0.
- ε -tube 범위를 벗어나면, 초과분만큼 손실로 계산.
- 모델이 ε 범위 내에 정확히 들어오도록 유도하되, 튜브 밖(과도하게 벗어난 샘플)에 대해서만 비용(슬랙 변수 $\xi, \hat{\xi}$)을 부여 $\rightarrow$ 오버피팅 방지 효과.

3 서포트 벡터 회귀

● ϵ -insensitive 손실 함수



출처:파이썬 프로그램 실습내용 중

3

서포트 벡터 회귀

● Support Vector Regression

$$y(x) = \mathbf{w}^T \phi(x) + b,$$

- $\phi(x)$: (고차원) 특징 공간으로 매핑
- $\mathbf{w}, b$: 회귀 초평면 파라미터

3

서포트 벡터 회귀

● ε -SVR의 프라이멀 문제

$$\min_{\mathbf{w}, b, \xi, \hat{\xi}} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N (\xi_n + \hat{\xi}_n)$$

$$\begin{aligned} \text{subject to} \quad & \xi_n \geq 0, \quad \hat{\xi}_n \geq 0, \quad n = 1, \dots, N, \\ & \xi_n \geq t_n - (\mathbf{w}^T \phi(x_n) + b) - \varepsilon, \quad n = 1, \dots, N, \\ & \hat{\xi}_n \geq (\mathbf{w}^T \phi(x_n) + b) - t_n - \varepsilon, \quad n = 1, \dots, N. \end{aligned}$$

- $\xi_n, \hat{\xi}_n$: 슬랙 변수(slack variables). ε -tube를 넘어서는 양쪽 오차들을 나타냄.
- $C > 0$: 모델의 복잡도(마진 폭)와 오차($\xi_n, \hat{\xi}_n$) 사이의 트레이드오프를 조절.

3

서포트 벡터 회귀

● ε -SVR의 라그랑지안

$$\begin{aligned}\mathcal{L}_P = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N (\xi_n + \hat{\xi}_n) - \sum_{n=1}^N \alpha_n (\varepsilon + \xi_n + y_n - t_n) \\ & - \sum_{n=1}^N \hat{\alpha}_n (\varepsilon + \hat{\xi}_n - y_n + t_n) - \sum_{n=1}^N (\mu_n \xi_n + \hat{\mu}_n \hat{\xi}_n)\end{aligned}$$

3 서포트 벡터 회귀

● ε -SVR의 KKT 조건

1. Stationarity 조건

$$\frac{\partial \mathcal{L}_P}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{n=1}^N (\alpha_n - \hat{\alpha}_n) \phi(x_n),$$

$$\frac{\partial \mathcal{L}_P}{\partial b} = 0 \implies \sum_{n=1}^N (\alpha_n - \hat{\alpha}_n) = 0,$$

$$\frac{\partial \mathcal{L}_P}{\partial \xi_n} = 0 \implies \alpha_n \in [0, C], \quad n = 1, \dots, N,$$

$$\frac{\partial \mathcal{L}_P}{\partial \hat{\xi}_n} = 0 \implies \hat{\alpha}_n \in [0, C], \quad n = 1, \dots, N.$$

3 서포트 벡터 회귀

● ε -SVR의 KKT 조건

2. Primal feasibility (프라이멀 타당성)

$$\left. \begin{aligned} \xi_n &\geq 0, & \hat{\xi}_n &\geq 0, \\ -\varepsilon - \xi_n - y_n + t_n &\leq 0, \\ -\varepsilon - \hat{\xi}_n + y_n - t_n &\leq 0, \end{aligned} \right\} \quad n = 1, \dots, N.$$

3. Dual feasibility (듀얼 타당성)

$$\alpha_n \geq 0, \quad \hat{\alpha}_n \geq 0, \quad \mu_n \geq 0, \quad \hat{\mu}_n \geq 0, \quad n = 1, \dots, N.$$

3

서포트 벡터 회귀

● ε -SVR의 KKT 조건

4. Complementary slackness (상호 보완성)

$$\left. \begin{aligned} \alpha_n(\varepsilon + \xi_n + y_n - t_n) &= 0, \\ \hat{\alpha}_n(\varepsilon + \xi_n - y_n + t_n) &= 0, \\ \mu_n \xi_n &= 0, \quad \hat{\mu}_n \hat{\xi}_n = 0, \end{aligned} \right\} \quad n = 1, \dots, N.$$

3 서포트 벡터 회귀

● ε -SVR의 듀얼 문제

$$\begin{aligned}
 \max_{\alpha, \hat{\alpha}} \quad & -\frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N (\alpha_n - \hat{\alpha}_n) (\alpha_m - \hat{\alpha}_m) k(x_n, x_m) \\
 & - \varepsilon \sum_{n=1}^N (\alpha_n + \hat{\alpha}_n) + \sum_{n=1}^N t_n (\alpha_n - \hat{\alpha}_n), \\
 \text{subject to} \quad & \sum_{n=1}^N (\alpha_n - \hat{\alpha}_n) = 0, \\
 & 0 \leq \alpha_n \leq C, \quad 0 \leq \hat{\alpha}_n \leq C, \quad n = 1, \dots, N.
 \end{aligned}$$

여기서,

- $\alpha_n, \hat{\alpha}_n$ 는 각각 ε -튜브 위·아래쪽 초과분을 보정하는 라그랑주 승수
- $k(x_n, x_m)$ 는 RBF 커널($\exp(-\gamma \|x_n - x_m\|^2)$)
- C 와 ε 은 하이퍼파라미터

출처:파이썬 프로그램 실습내용 중

3 서포트 벡터 회귀

● ε -SVR의 듀얼 문제

$$\min_{\alpha, \hat{\alpha}} \quad \frac{1}{2}(\alpha - \hat{\alpha})^T K_{n,m}(\alpha - \hat{\alpha}) + \begin{bmatrix} \varepsilon \mathbf{1}_N \\ \varepsilon \mathbf{1}_N \end{bmatrix}^T \begin{bmatrix} \alpha \\ \hat{\alpha} \end{bmatrix} - \begin{bmatrix} \mathbf{t} \\ -\mathbf{t} \end{bmatrix}^T \begin{bmatrix} \alpha \\ \hat{\alpha} \end{bmatrix},$$

$$\text{subject to} \quad \begin{bmatrix} \mathbf{1}_N \\ \mathbf{1}_N \end{bmatrix}^T \begin{bmatrix} \alpha \\ \hat{\alpha} \end{bmatrix} = 0,$$

$$\begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix} \leq \begin{bmatrix} \alpha \\ \hat{\alpha} \end{bmatrix} \leq \begin{bmatrix} C \\ \vdots \\ C \end{bmatrix}.$$

3

서포트 벡터 회귀

● ε -SVR의 결정 함수

$$y(x) = \sum_{n=1}^N (\alpha_n - \hat{\alpha}_n) k(x_n, x) + b,$$

3 서포트 벡터 회귀

● ε -SVR 에서 b 구하기

$$b = t_n - \varepsilon \operatorname{sgn}(\alpha_n - \hat{\alpha}_n) - \sum_{m=1}^N (\alpha_m - \hat{\alpha}_m) k(x_m, x_n).$$

여기서,

- $\operatorname{sgn}(\alpha_n - \hat{\alpha}_n)$ 은 " $\alpha_n > 0$ 이면 -1", " $\hat{\alpha}_n > 0$ 이면 +1" 이런식으로 튜브 아래·위 구분을 반영하는 부호 함수이다.
- 실제 구현에서는, $0 < \alpha_n < C$ 또는 $0 < \hat{\alpha}_n < C$ 인 서포트 벡터들을 찾아서 해당 점들에 대해 KKT 조건으로부터 b 의 평균을 사용한다.

4

서포트 벡터 회귀 실습



4 SVR 파이썬 실습

● 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# 데이터 생성 (2차 다항식)
np.random.seed(42)
N = 30
X = np.linspace(-3, 3, N).reshape(-1, 1)  # (N,1) shape

# 2차식:  $y_{true} = 0.5 * x^2 + 0.5 * x + 1.0$ 
y_true = 0.5 * (X ** 2).squeeze() + 0.5 * X.squeeze() + 1.0

# 노이즈 추가
noise = np.random.normal(0, 1.0, size=N)
t = y_true + noise
```

출처:파이썬 프로그램 실습내용 중

4 SVR 파이썬 실습

● RBF 커널 함수

```
def rbf_kernel(x1, x2, gamma=0.5):  
    """RBF 커널:  $\exp(-\gamma * ||x1 - x2||^2)$ """  
    # x1, x2가 shape (1,)라고 가정  
    diff = float(x1[0] - x2[0]) # -> 스칼라 변환  
    return np.exp(-gamma * (diff**2))  
  
def build_kernel_matrix(X, gamma=0.5):  
    """X(1D): shape (N,1) -> NxN RBF 커널 행렬"""  
    N = len(X)  
    K = np.zeros((N, N))  
    for i in range(N):  
        for j in range(N):  
            K[i, j] = rbf_kernel(X[i], X[j], gamma=gamma)  
    return K
```

출처:파이썬 프로그램 실습내용 중

4 SVR 파이썬 실습

● 듀얼 목적 함수

```
def svr_dual_objective(a, t, K, epsilon):
    # a = [alpha_1, ..., alpha_N, alpha^_1, ..., alpha^_N]
    N = len(t)
    alpha = a[:N]
    alpha_hat = a[N:]
    # 1)  $0.5 * \sum_{i,j} (\alpha_i - \alpha_{\hat{i}})(\alpha_j - \alpha_{\hat{j}})K[i,j]$ 
    diff = alpha - alpha_hat
    term1 = 0.5 * np.sum((diff[:,None] * diff[None,:]) * K)

    # 2)  $\epsilon * \sum_i (\alpha_i + \alpha_{\hat{i}})$ 
    term2 = epsilon * np.sum(alpha + alpha_hat)

    # 3)  $-\sum_i y_i (\alpha_i - \alpha_{\hat{i}})$ 
    term3 = - np.sum(t * diff)

    return term1 + term2 + term3
```

출처:파이썬 프로그램 실습내용 중

4 SVR 파이썬 실습

● 제약조건 함수

```
def constraint_sum_alpha(a, t):  
    """sum_i (alpha_i - alpha^_i) = 0"""  
    N = len(t)  
    alpha = a[:N]  
    alpha_hat = a[N:]  
    return np.sum(alpha - alpha_hat)
```

4 SVR 파이썬 실습

● RBF SVR 학습 함수

```
def fit_epsilon_svr_rbf(X, t, C, epsilon, gamma):  
    N = len(X)  
    K = build_kernel_matrix(X, gamma=gamma)  
  
    # 초기값  
    alpha_init = np.zeros(N)  
    alpha_hat_init = np.zeros(N)  
    vars_init = np.concatenate([alpha_init, alpha_hat_init])  
  
    # 제약 조건 (등식 제약)  
    eq_cons = {'type': 'eq', 'fun': constraint_sum_alpha, 'args': (t,)}  
  
    # alpha, alpha^* 범위 지정 ([0, C])  
    bounds = [(0, C)] * (2*N)
```

출처:파이썬 프로그램 실습내용 중

4 SVR 파이썬 실습

● RBF SVR 학습 함수

```
res = minimize(  
    fun=svr_dual_objective,  
    x0=vars_init,  
    args=(t, K, epsilon),  
    constraints=[eq_cons],  
    bounds=bounds,  
    options={'maxiter': 500, 'disp': True}  
)  
alpha_opt = res.x[:N]  
alpha_hat_opt = res.x[N:]  
return alpha_opt, alpha_hat_opt, K
```

4 SVR 파이썬 실습

● b 계산 함수

```
def compute_b(alpha, alpha_hat, X, t, K, epsilon, C):  
    N = len(X)  
    diff = alpha - alpha_hat  
  
    # 서포트 벡터(SV) 후보:  $\alpha_i, \alpha_i^*$ 가  $(0, C)$  범위 안에 위치한 지점  
    sv_candidates = [  
        i for i in range(N)  
        if (1e-8 < alpha[i] < C - 1e-8) or (1e-8 < alpha_hat[i] < C - 1e-8)  
    ]  
  
    if not sv_candidates:  
        # 서포트 벡터가 없으면 0 반환 (예외적 상황)  
        return 0.0
```

4 SVR 파이썬 실습

● b 계산 함수

```
# 여러 SV 후보들에 대해 b 값을 구하고, 그 평균값을 최종 b로 사용
b_values = []
for i in sv_candidates:
    sign_val = np.sign(diff[i]) # (alpha_i - alpha_hat_i)의 부호
    sum_j = np.sum(diff * K[i, :]) #  $\sum_j (\alpha_j - \alpha_{\hat{j}}) * K[i, j]$ 
    b_i = t[i] - epsilon * sign_val - sum_j
    b_values.append(b_i)

# b의 평균
b_est = np.mean(b_values)
return b_est
```

4 SVR 파이썬 실습

● 예측 함수

```
def predict_svr(alpha, alpha_hat, b, X_train, X_test, K, gamma=0.5):  
    """  
     $y(x) = \sum_n (\alpha_n - \alpha_n^{\text{hat}}) * K(x_n, x) + b$   
    """  
    diff = alpha - alpha_hat  
    y_pred = []  
    for x in X_test:  
        val = b  
        for i, x_n in enumerate(X_train):  
            val += diff[i] * np.exp(-gamma * (x_n - x) * (x_n - x))  
        y_pred.append(val)  
    return np.array(y_pred)
```


4 SVR 파이썬 실습

● SVR 실행 : (C=10 / eps=1 / gamma=0.5) 튜닝 가능

```
alpha_opt, alpha_hat_opt, K = fit_epsilon_svr_rbf(X, t,
                                                    C=C,
                                                    epsilon=eps,
                                                    gamma=gamma)

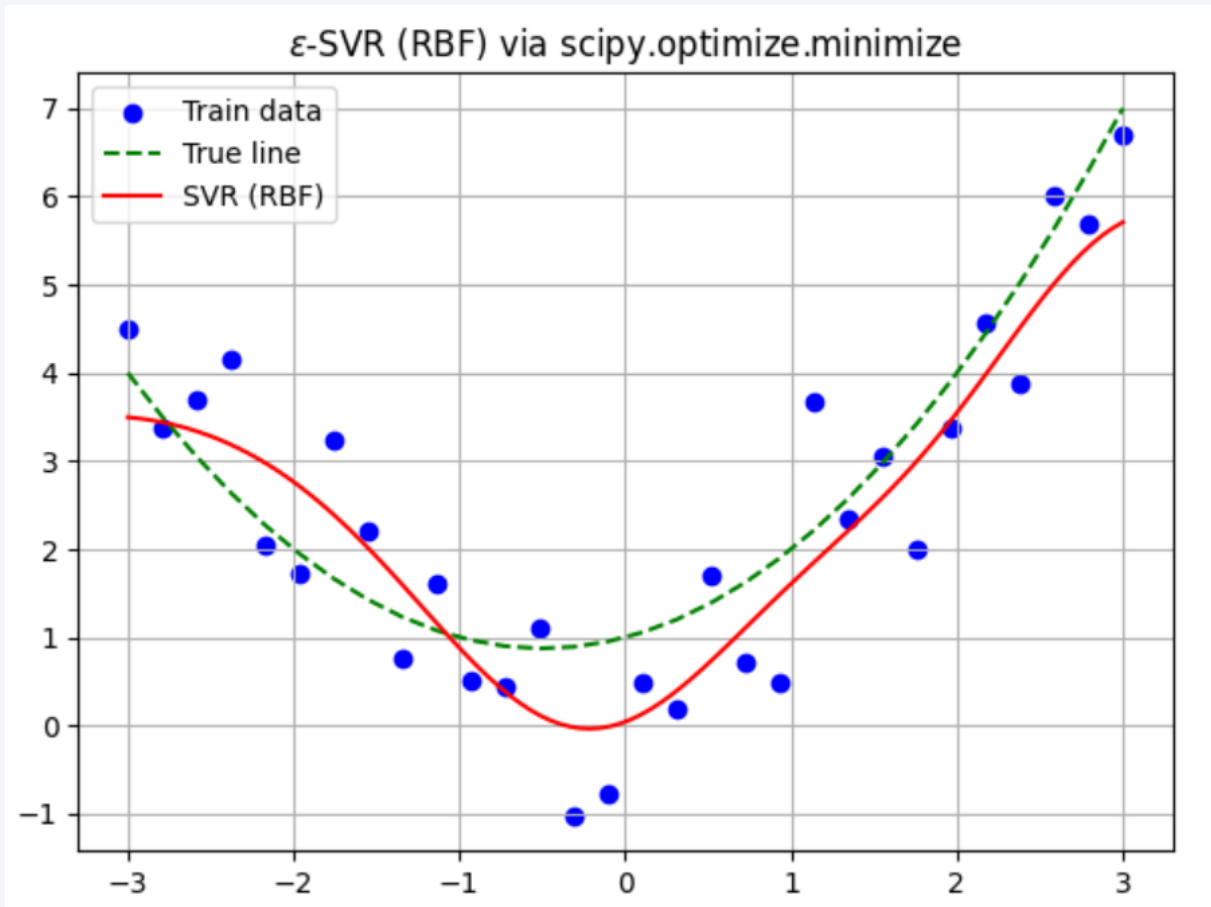
print("[Optimization] alpha_opt:", alpha_opt)
print("[Optimization] alpha_hat_opt:", alpha_hat_opt)

b_opt = compute_b(alpha_opt, alpha_hat_opt, X, t, K, eps, C)
print("[Optimization] b_opt:", b_opt)

# 예측
X_test = np.linspace(-3, 3, 100).reshape(-1,1)
y_pred = predict_svr(alpha_opt, alpha_hat_opt, b_opt, X, X_test, K, gamma=gamma)
```

4 SVR 파이썬 실습

● 결과 시각화



출처: 파이썬 프로그램 실습내용 중

★ 정리하기

■ Soft-Margin SVM

슬랙 변수 : 오분류와 마진 침범을 허용

Soft-Margin SVM : 슬랙 변수를 도입한 SVM / 일반화 개선

■ Support Vector Regression (SVR)

SVM을 회귀 문제로 확장

ϵ -insensitive loss 도입 : 예측 오차가 ϵ 보다 작으면 손실 0

하이퍼 파라미터 γ, C, ϵ 값을 조절하여 SVR 성능 튜닝 가능

13 강

다음시간안내

커널 기법 3

